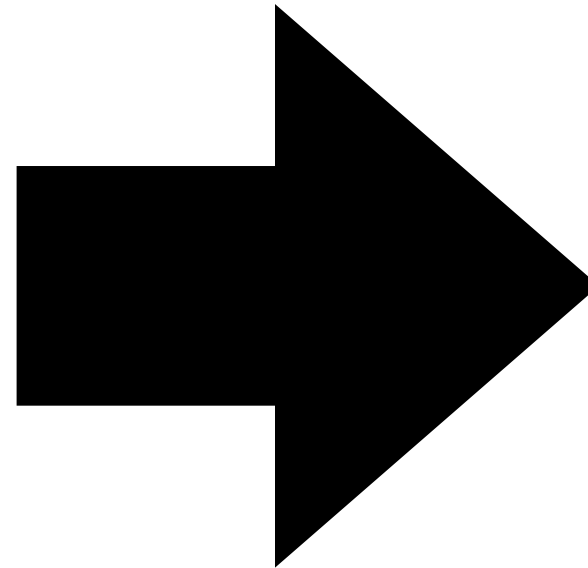
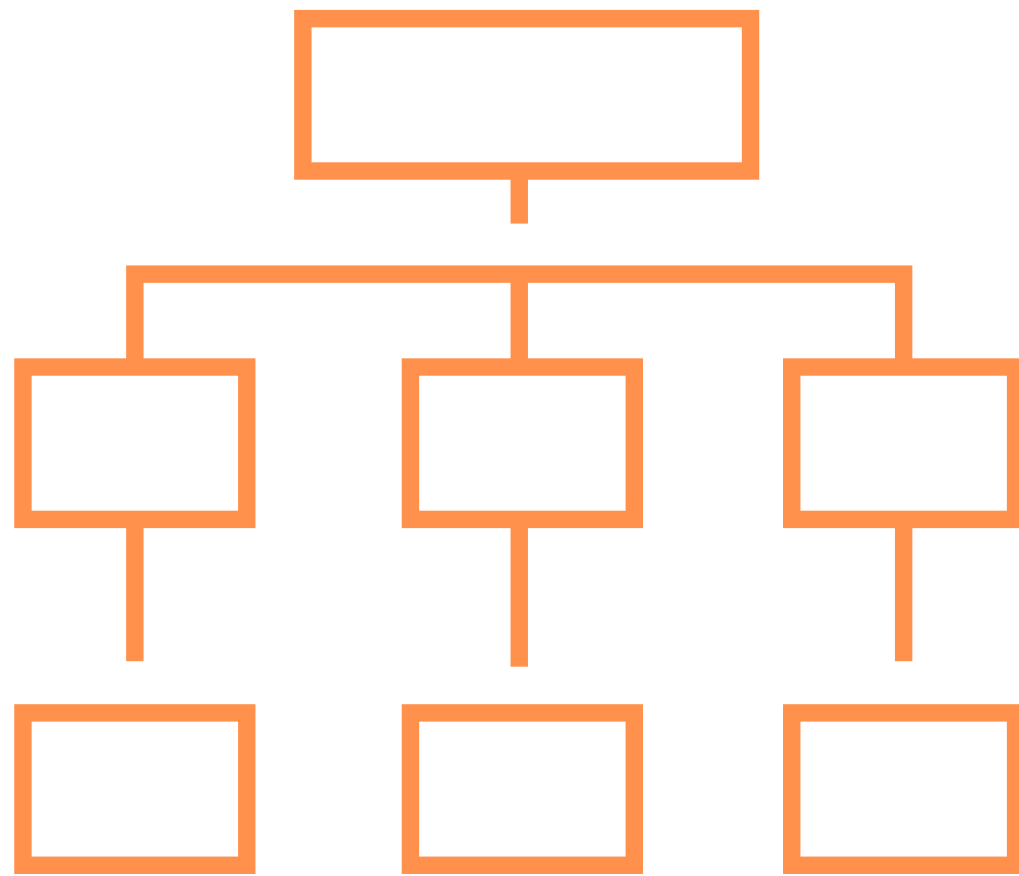


CMP5354

Software Design

Linking UML to Code



```
public class UMLtoCode {  
    private List<String> mappings;  
    private HashMap<String> example;
```

```
    public UMLtoCode(List<String> m, HashMap<String> e ) {
```

```
        this.mappings=m;  
        this.example = e;
```

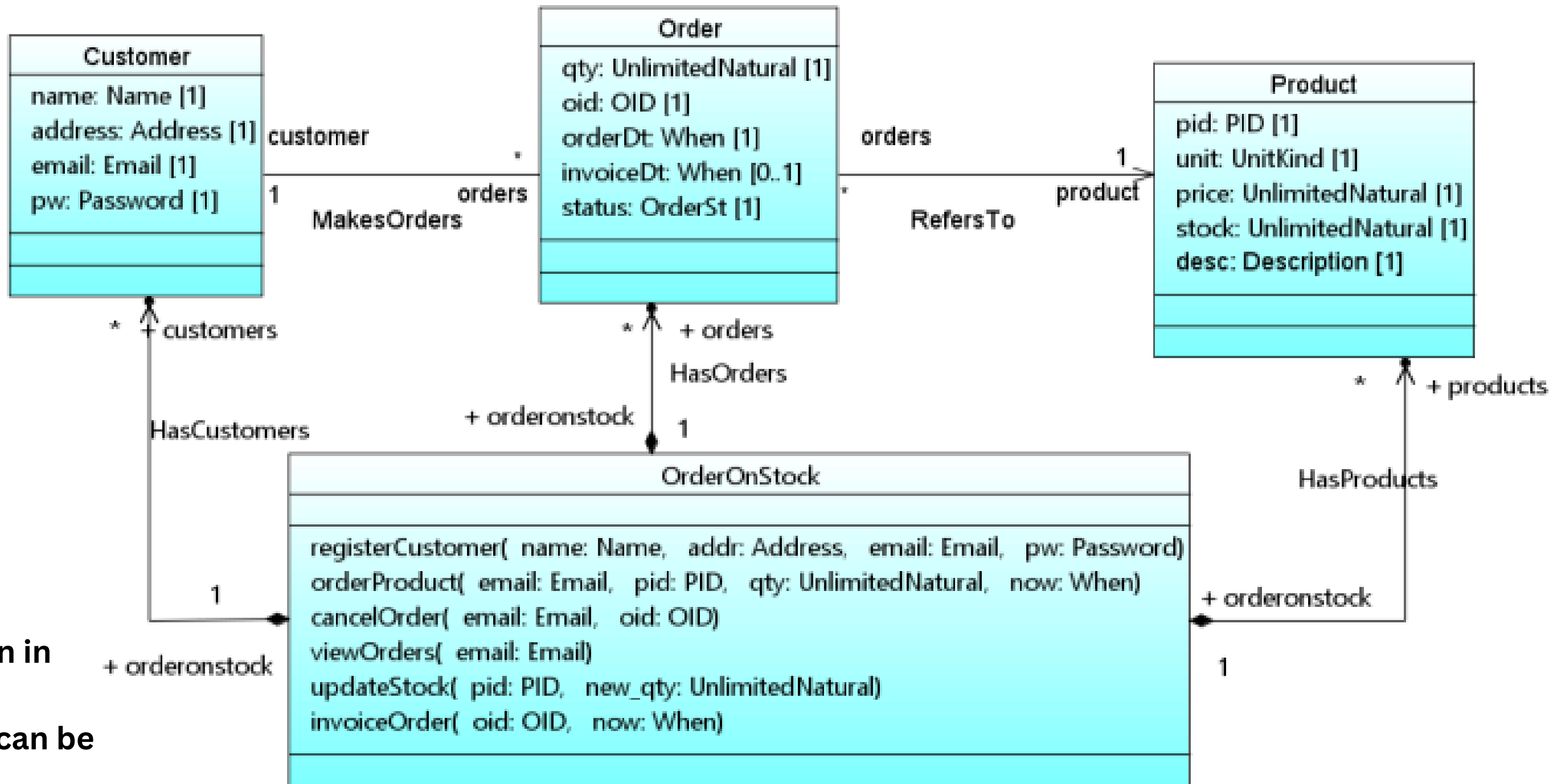
```
    }  
}
```

UML to Code Mapping

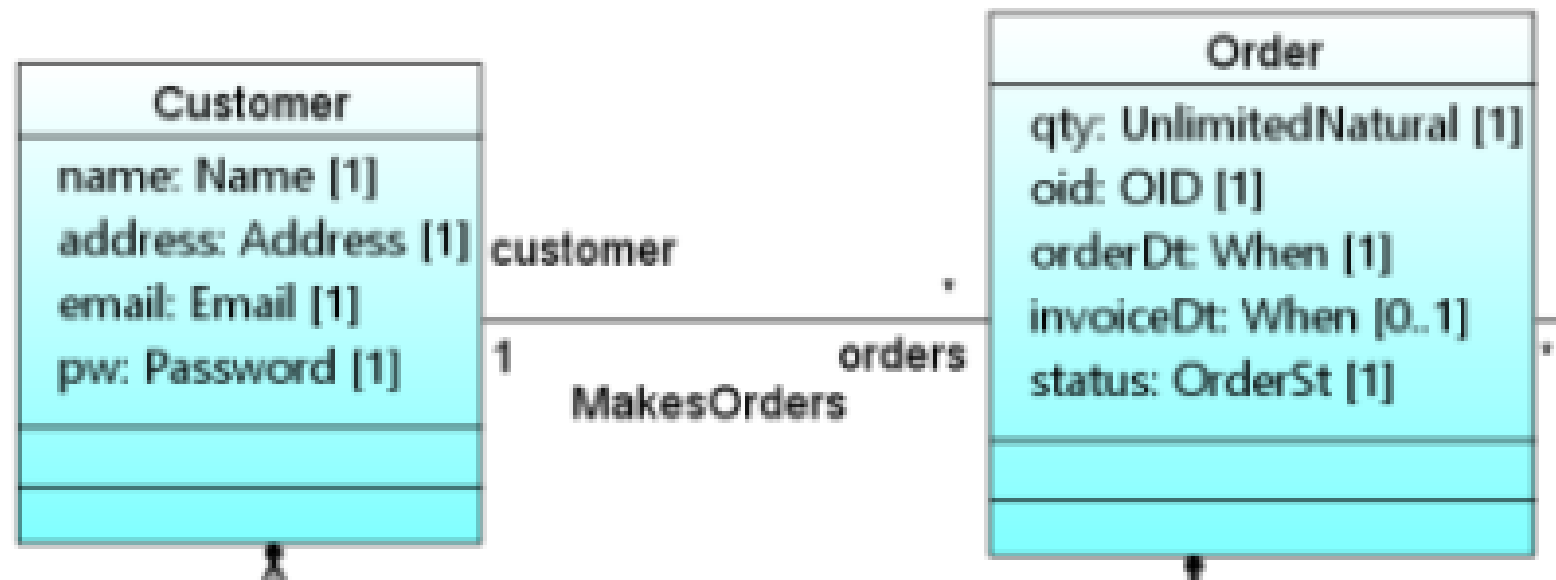
<u>UML</u>	<u>Code Constructs</u>
UML diagrams	Object-oriented code (e.g. Java, Python, C)
Classes	Classes
Attributes	Attributes with their data types
Associations, aggregations, compositions	Attributes, method calls
Inheritance	Super classes and sub-classes
Sequence Diagrams	Methods

Mapping Examples

Partial Class Diagram



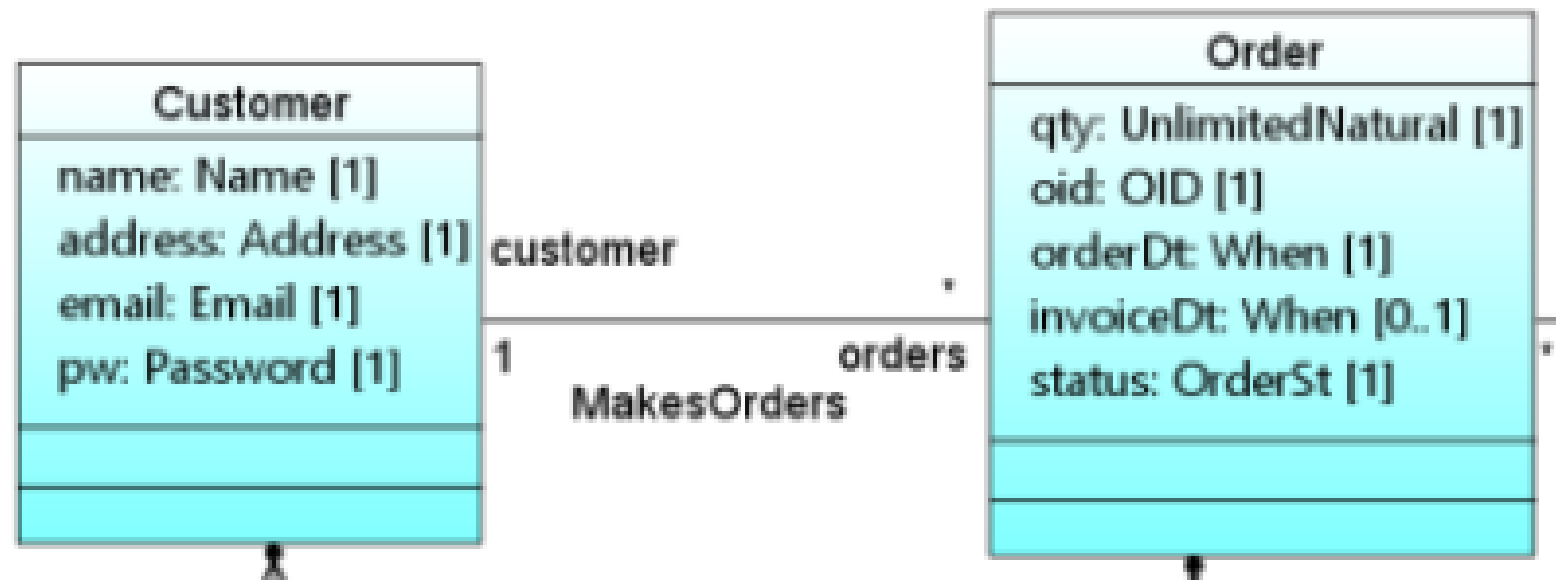
Alternative notation in
this class diagram:
generic data types can be
written if the
programming language is
still not set in stone.



```
class Customer {
    String name;
    Address address;
    String email;
    String password;
    List<Order> orders;
}
```

The type of the attribute can be another class that you will implement

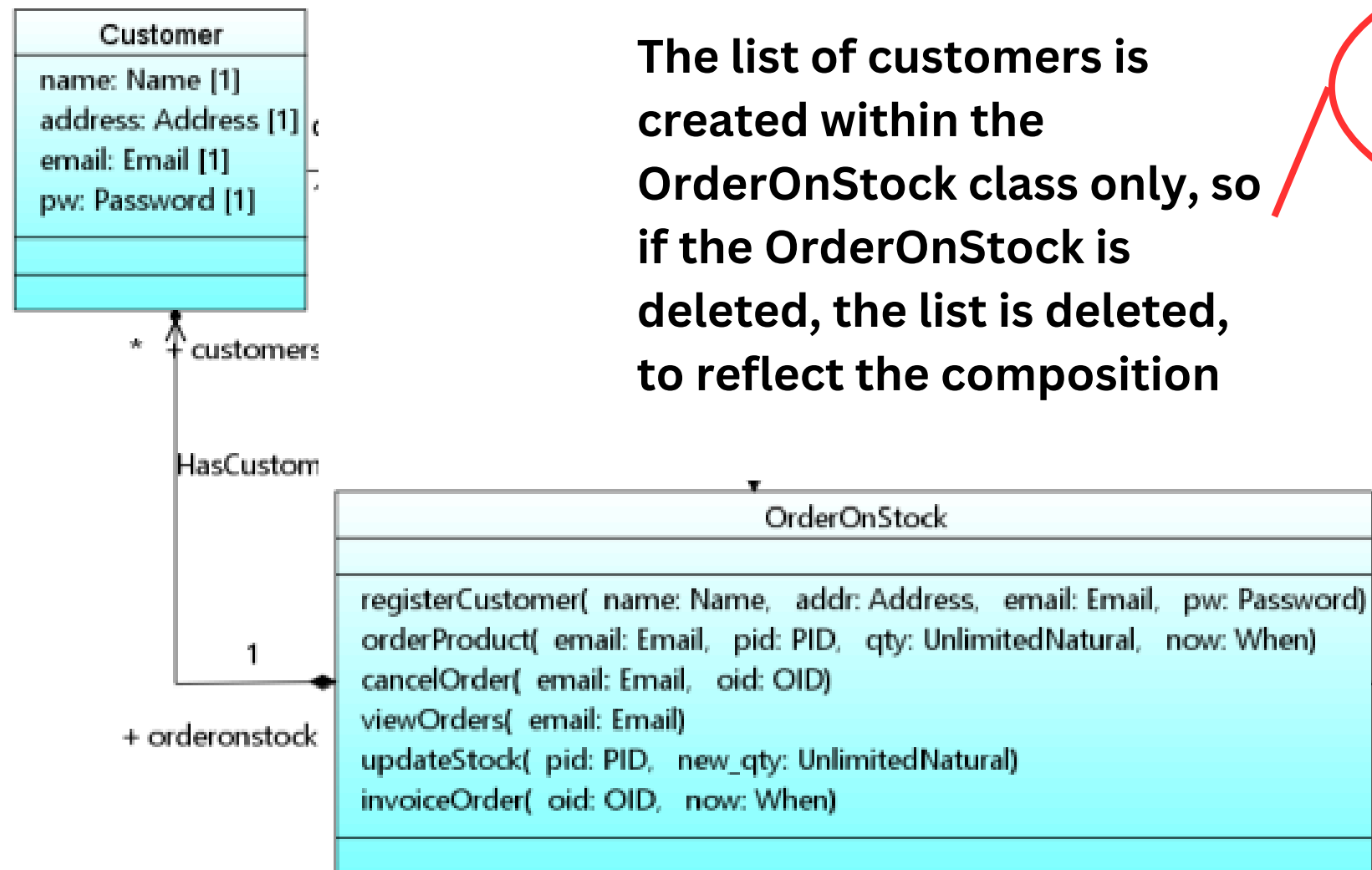
The attributes can be derived from associations



```
class Order {  
    String oid;  
    int qty;  
    LocalDate orderDt;  
    LocalDate invoiceDt;  
    Customer customer;  
}
```

The attributes can be
derived from associations

From OrderOnStock class

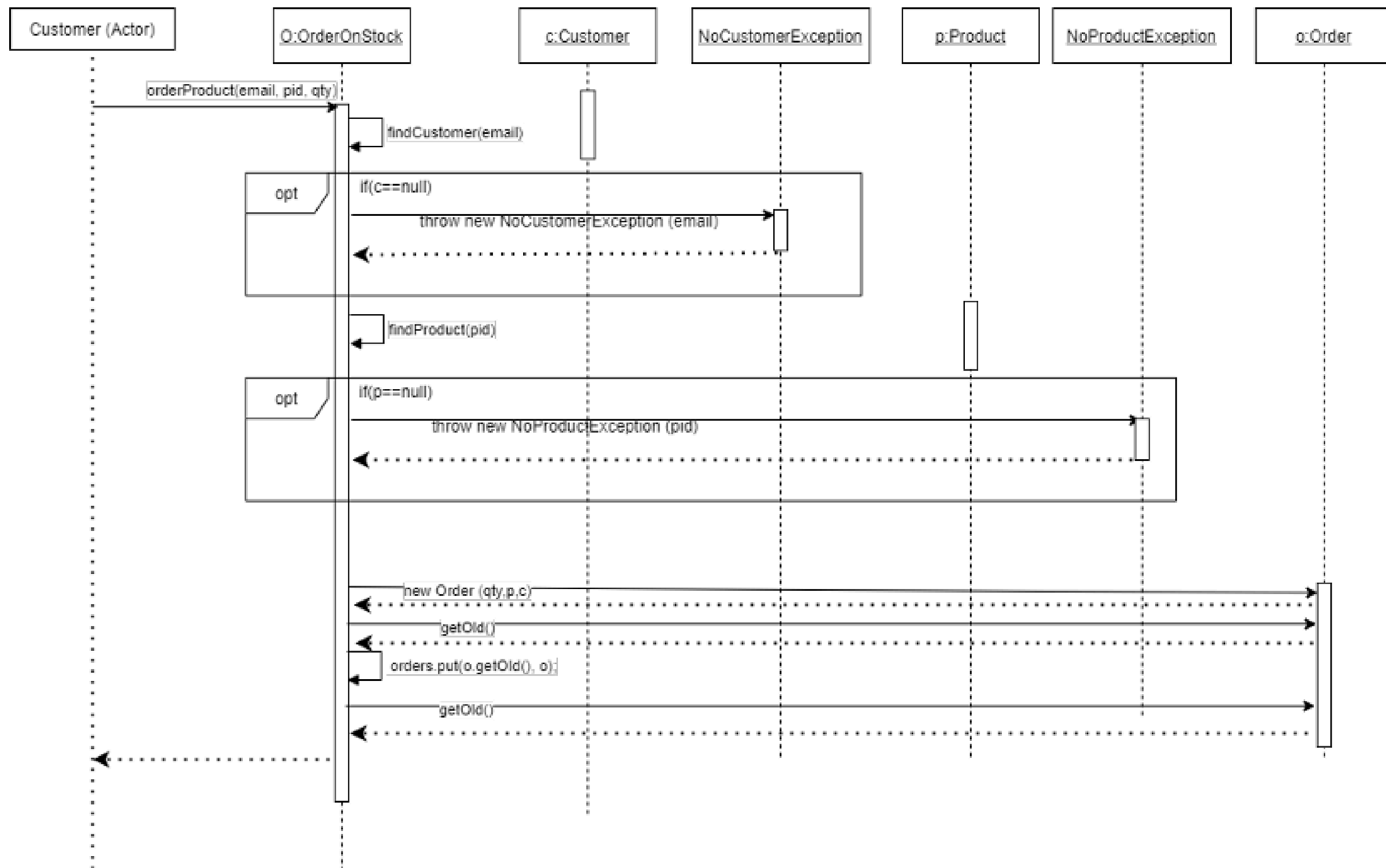


The list of customers is created within the **OrderOnStock** class only, so if the **OrderOnStock** is deleted, the list is deleted, to reflect the composition

```
class OrderOnStock:
    def __init__(self):
        # Composed customers – their lifecycle is tied to OrderOnStock
        self.customers = []

    def register_customer(self, name, address, email, password):
        customer = Customer(name, address, email, password)
        self.customers.append(customer)
        print(f"Registered: {customer}")

    def list_customers(self):
        for customer in self.customers:
            print(customer)
```



From OrderOnStock class

```
public String orderProduct (String email, String pid, int qty)
    throws NoCustomerException, NoProductException {

    Customer c = findCustomer(email);

    if (c == null)
        throw new NoCustomerException (email);

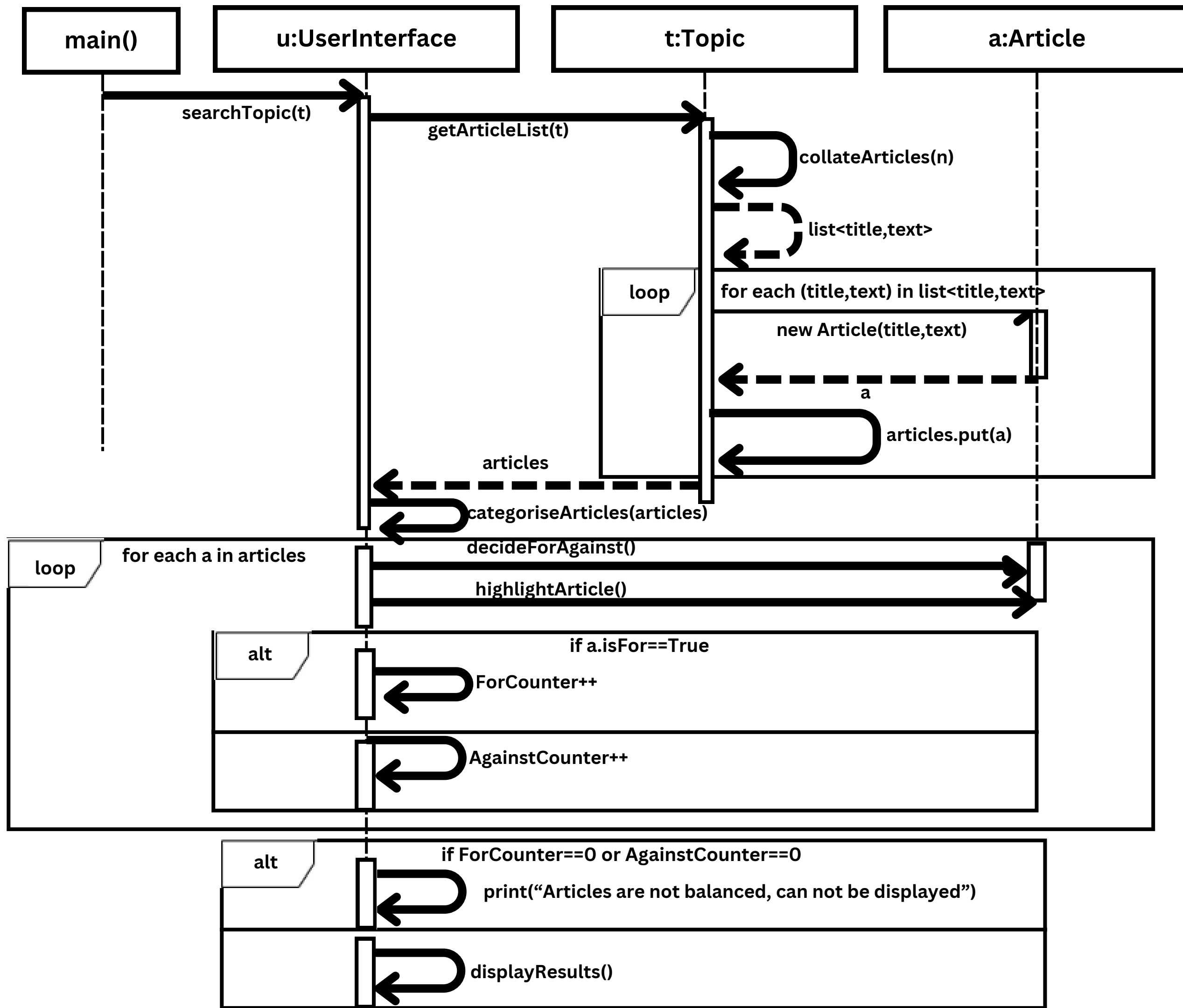
    Product p = findProduct(pid);

    if (p == null)
        throw new NoProductException(pid);

    Order o = new Order(qty, p, c);

    orders.put(o.getId(), o);

    return (o.getId());
}
```



From UserInterface class

```
public void searchTopic(String t) {  
    Topic topic = new Topic(t);  
    List<Article> articles = topic.getArticleList(t);  
    categoriseArticles(articles);  
    displayResults(articles);  
}  
  
public void categoriseArticles(List<Article> articles) {  
    int forCounter = 0;  
    int againstCounter = 0;  
  
    for (Article a : articles) {  
        a.decideForAgainst();  
        a.highlightArticle();  
  
        if (a.isFor()) {  
            forCounter++;  
        } else {  
            againstCounter++;  
        }  
    }  
  
    if (forCounter == 0 || againstCounter == 0) {  
        System.out.println("Articles are not balanced, cannot be displayed");  
        return;  
    }  
}
```



From Topic class

```
public class Topic {  
    private String name;  
    private List<Article> articles;  
    private String id;  
  
    public Topic(String name) {  
        this.name = name;  
        this.articles = new ArrayList<>();  
        this.id = generateId();  
    }  
}
```

From Topic class - continued

```
public List<Article> getArticleList(String t) {  
    Map<String, String> collatedArticles = collateArticles(t);  
  
    for (Map.Entry<String, String> entry : collatedArticles.entrySet()) {  
        articles.add(new Article(entry.getKey(), entry.getValue()));  
    }  
    return articles;  
}
```

```
private Map<String, String> collateArticles(String n) {  
    Map<String, String> articlesMap = new HashMap<>();  
  
    // Example static articles, replace with real data fetching logic  
    articlesMap.put("Title 1", "Content of article 1.");  
    articlesMap.put("Title 2", "Content of article 2.");  
  
    return articlesMap;  
}
```

```
}
```


From Article class

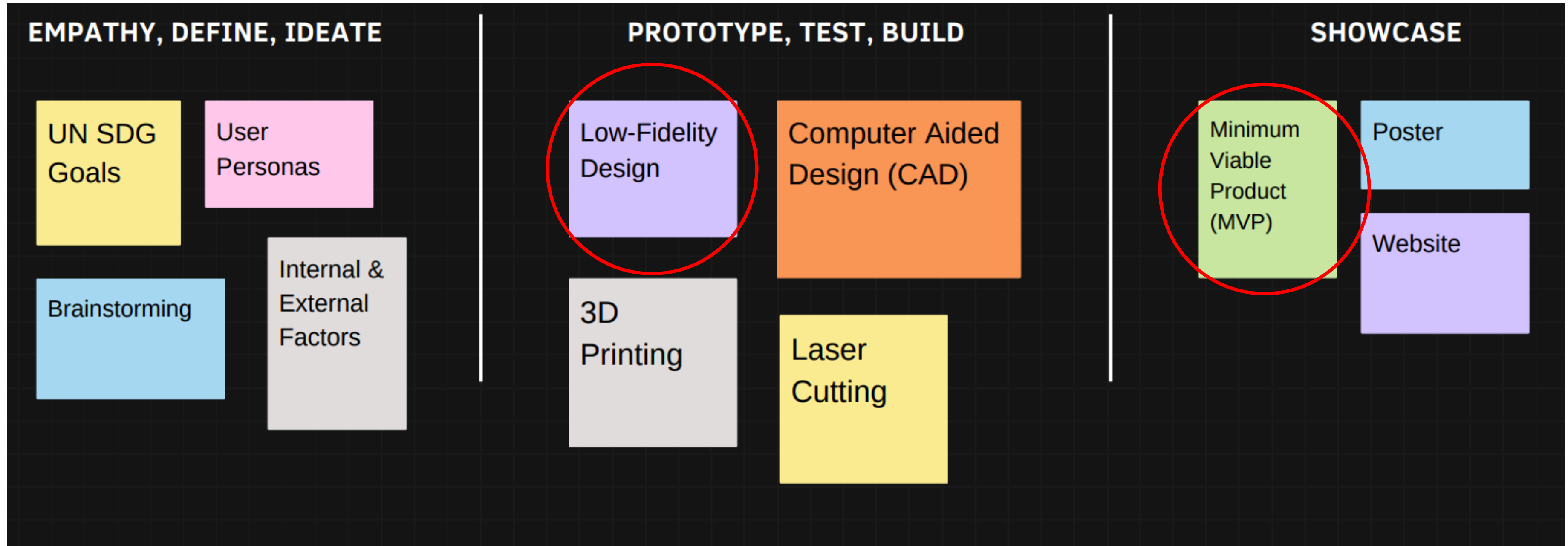
```
public class Article {  
    private String title;  
    private String text;  
    private boolean isFor;  
    private String highlight;  
  
    public Article(String title, String text) {  
        this.title = title;  
        this.text = text;  
        this.isFor = false; // Default value  
        this.highlight = "amber"; // Default highlight  
    }  
}
```

From Article class - continued

```
public boolean isFor() {  
    return isFor;  
}  
  
public void decideForAgainst() {  
    // Example Logic: articles with "support" are considered "for"  
    if (text.toLowerCase().contains("support")) {  
        isFor = true;  
    } else {  
        isFor = false;  
    }  
}  
  
public void highlightArticle() {  
    // Example Logic: highlight "red" if against, "amber" if for  
    if (isFor) {  
        highlight = "amber";  
    } else {  
        highlight = "red";  
    }  
}
```

What to include for MVP submissions?

Which module is this slide from?



Linking UML to MVPs

UML
UML diagrams
Classes
Attributes
Associations, aggregations, compositions
Inheritance
Sequence Diagrams

Low-Fidelity MVP



(High-Fidelity MVP)
Object-oriented code (e.g. Java, Python, C)
Classes
Attributes with their data types
Attributes, method calls
Super classes and sub-classes
Methods